V8のTorqueを語りたい

西 悠太　/ 株式会社ダイニー

(Browser and UI #4 Anyの替え玉)

自己紹介

西 悠太 (Nishi Yuta)

21歳(重要、important)

V8 Contributor

Platform Engineer at Dinii Inc.

TSKaigi Staff

TypeScriptが好きです
V8にパッチを送るのも好きです
あと野球も好きです

@riya-amemiya



V8とは？

V型8気筒（ブイがたはちきとう）は、レシプロエンジン等のシリンダー配列形式の一つで、直列4気筒2組がV字様に配置されている形式を指す。当記事では専らピストン式内燃機関のそれについて述べる注釈 1。V8（ブイはち）と略されることが多い。

多気筒レシプロエンジンとして広く用いられるエンジン形式の一つであり、自動車用としては特に大排気量車の多かったアメリカ合衆国で発達してきた。ガソリンエンジン、ディーゼルエンジン双方あるも、現代では大型乗用車用のエンジン形式として普及している。 引用: V型8気筒 - Wikipedia

V8とは？

V8は、Googleが開発するオープンソースのJIT仮想マシン型のJavaScriptエンジンである3。この名前は同じく「V8」と略されるV型8気筒エンジンに由来している4。Google ChromeなどのChromiumベースのブラウザや、Node.jsなどで採用されている。

概要
ECMAScript (ECMA-262) 準拠で、C++で記述されている。スタンドアローンでの実行が可能なほか、C++で書かれたアプリケーションの一部として動作させることもできる。

引用: V8 (JavaScriptエンジン) - Wikipedia

「V8はC++で書かれている」

確かにほとんどの実装はC++です。

builtinsをのぞいてみると、そこにあるのは
C++でもJavaScriptでもない、見慣れない言語。

Torque

実際にTorqueを見てみる

```
// src/builtins/array-foreach.tq より抜粋
transitioning javascript builtin ArrayForEach(
    js-implicit context: NativeContext, receiver: JSAny)(
    ...arguments): JSAny {
  const o: JSReceiver = ToObject_Inline(context, receiver);
  const len: Number = GetLengthProperty(o);
  const callbackfn = Cast<Callable>(arguments[0])
      otherwise TypeError;
  // ...
}
```

なぜGoogleは、
V8専用の言語を
わざわざ作ったのか？

Torque以前
CSAの時代

builtinsは手書きされていた

かつてV8のbuiltinsは、CodeStubAssembler（CSA）というC++ベースの低水準な記法で手書きされていました。

CSAは速い。けれど…

CSAの強み

- → 関数呼び出しの余計なコストを避けられる
- → 機械語レベルで攻めた最適化ができる
- → 1つの記述で全アーキ向けに生成できる

CSAの弱み

- → 型の保証が弱い
- → 制御フローが手続き的で読みにくい
- → 正しく書くのに深い専門知識が要る

一番の問題は、型の保証が弱いこと

```
TNode<Object> elem = LoadFixedArrayElement(elements, index);

TNode<JSArray> arr = UncheckedCast<JSArray>(elem);
```

CSAにも `TNode<T>` という型はあります。
ただ無検査のキャストが通るため、 `elem` が本当に JSArray かまでは保証されません。

取り違えは、脆弱性になる

型の取り違えが容易に起こり、しかもそれが
クラッシュや深刻な脆弱性の原因になっていました。

型ごとにメモリ上の配置が違うため、誤った型として読むと、
無関係な領域をフィールドとして読み書きしてしまうためです。

つまり、こういう状況

安全で速いコードをCSAで書くには、
多くの専門知識が求められていました。

Torqueの登場
型のある世界へ

Torque = 静的型付きのDSL

CSA

- × 型の保証が弱い
- × 無検査キャストで取り違えが起きうる
- × 専門知識が前提

Torque

- ○ V8の値に静的な型が付く
- ○ 取り違えをコンパイル時に検出しやすい
- ○ 仕様に近い書き方ができる

仕様の手順と対応づけて書ける

ECMAScript 仕様 (forEach)

1. Let `O` be ? ToObject(this value).
2. Let `len` be ? LengthOfArrayLike(O).
3. If IsCallable(callbackfn) is false, throw a **TypeError** exception.

Torque

```
const o: JSReceiver =
    ToObject_Inline(context, receiver);
const len: Number =
    GetLengthProperty(o);
const callbackfn =
    Cast<Callable>(arguments[0])
    otherwise TypeError;
```

失敗経路を、コンパイル時に強制

```
const arr = Cast<JSArray>(obj) otherwise Bailout;

const n: Number = x;
typeswitch (n) {
  case (s: Smi): { ... }
  case (h: HeapNumber): { ... }
}
```

`Cast` は実行時に型を検査し、**失敗経路を書かないとコンパイルが通りません**。
`typeswitch` は共用型を型ごとに分け、網羅性まで検査されます。

でも、
速さはどうなる？

Torqueはコンパイルされる



Torqueは実行時に解釈される言語ではありません。
CSA **を使う** C++ にコンパイルされ、最終的に機械語になります。

だから、速さを犠牲にしない

Torqueは**読みやすさ**と**型安全**を足しながら、
CSAの性能を引き継ぎます。

では、Torqueの強さはどこで発揮されるか

CSA の操作を、extern macro で呼ぶ

```
// src/builtins/js-array.tq
extern macro MoveElements(
    FixedArray|FixedDoubleArray, Smi, Smi, Smi): void;
```

`extern macro` は、CSA 側の実装を Torque から呼ぶ宣言です。
`MoveElements` の実体は CodeStubAssembler にあります。

同じ処理を CSA で手書きし直す必要がありません。
Runtime を経由せず、生成コード側で完結させられます。

最適化後も、Torque builtin が使われる

V8 では、Maglev や TurboFan が生成するコードからも
Torque で書いた builtin を呼ぶことがあります。

一方で、TurboFan が forEach をインライン展開するように、
最適化側が別経路を持つ場合もあります。

それでも、Torque で書いた builtin は
インタプリタ実行や、インライン化されない呼び出しの土台になります。

まとめ

Torqueが解きたかった問題

読みやすさ

仕様の手順と対応づけて書ける

安全性

型の取り違えをコンパイル時に検出しやすい

速度

CSA の性能を Torque から使える

この3つを同時に満たすために、Googleは専用言語という重い選択をしました。

今日覚えて帰ってほしいこと

- → V8 の builtins の多くは Torque という専用言語で書かれている
- → CSA の手書きは速いが、**型の保証が弱かった**
- → Torque は静的型付きで、**取り違えをコンパイル時に検出しやすい**
- → CSA を使う C++ にコンパイルされるため、**速さを犠牲にしない**
- → `extern macro` で CSA の操作を呼べる
- → 最適化後も Torque builtin が使われることが多い（インライン化など例外あり）

参考資料

- V8 Docs: Torque user manual
- V8 Docs: Torque builtins
- V8 Blog: Taming architecture complexity in V8 — the CodeStubAssembler

ご清聴ありがとうございました